UNIVERSIDAD DE SAN CARLOS DE GUATEMALA
Centro Universitario del Norte
Ingeniería en Ciencias y Sistemas

Memoria final
C Extended

Analizador léxico y sintáctico
Lenguajes Formales de Programación
Entregable 5 - Pruebas, documentación y sustentación final

Integrante	Carné
Cristian Gabriel Ramírez Morales	202540362
Enrique José Sosa Caal	202544363
José Roberto Posadas Ascencio	202544488

Docente: Ing. José Alberto Veliz Cruz
Entrega: 3 de noviembre de 2026

Repositorio: github.com/CGRM2/C-EXTENDED

Base verificada actual del codigo: `88216455dc6a2ab224b3affb575a1258786b3efe`

Introducción y objetivos

C-Extended es una aplicación de escritorio que reconoce las unidades léxicas de un lenguaje académico inspirado en C# y presenta sus ubicaciones, identificadores y errores. La versión final incorpora además un analizador sintáctico de la gramática simplificada del proyecto. Este documento consolida la especificación, el diseño, la implementación, las pruebas y los manuales que corresponden a esa versión.

La aplicación permite escribir código o cargar archivos de texto, analizar su contenido y exportar un informe. El reconocimiento léxico se implementa carácter por carácter. Las expresiones regulares se utilizan como descripción formal del lenguaje, sin recurrir a un motor Regex ni a un generador de analizadores para reconocer la entrada.

Objetivo general

Desarrollar y verificar un analizador de C-Extended que aplique fundamentos de lenguajes formales y ofrezca diagnósticos comprensibles mediante una interfaz Windows Forms.

Objetivos específicos

• Definir las categorías de tokens y las reglas de formación de lexemas del lenguaje.

• Relacionar los reconocedores manuales con sus estados, transiciones y condiciones de aceptación.

• Registrar errores con línea y columna y recuperar la lectura cuando el contexto lo permite.

• Integrar tokens, errores, símbolos, declaraciones y estadísticas a partir del mismo texto analizado.

• Verificar entradas válidas, límites, errores combinados y funciones de integración mediante pruebas reproducibles.

Resultado de la verificación

La ejecución final aprobó 22 casos documentados, 76 comprobaciones de regresión léxica y 125 de sintaxis e integración. Son 223 comprobaciones/casos aprobados en tres baterías distintas. Este resultado acredita los escenarios ejecutados; no constituye una prueba de ausencia de fallos para toda entrada posible.

Continuidad de los cinco entregables

Etapa	Contenido consolidado en esta versión
1 Especificación	Lenguaje C-Extended, alfabeto, categorías de tokens, reglas de error y tabla de símbolos.
2 Diseño	Reconocedores manuales, transiciones, responsabilidades de clases y separación entre Core y UI.
3 Motor	Recorrido del texto, posiciones base 1, máximo avance, tokens, recuperación y pruebas del núcleo.
4 Integración	Editor, carga, tablas, estadísticas, navegación, exportación y manual de uso.
5 Consolidación	Matriz ampliada, evidencia actual, manual técnico y de usuario, memoria y sustentación.

Cambios frente al planteamiento inicial

La lista original de reservadas se amplió a 30 palabras predeterminadas. Se incluyeron caracteres literales, operadores lógicos y escapes. El reconocimiento de números inválidos consume la secuencia problemática sin fragmentarla en tokens engañosos. Se documentó que los comentarios de bloque cierran en el primer */ y no se anidan.

La interfaz final añadió un diccionario persistente de clasificaciones y una etapa sintáctica. Ambas son extensiones del proyecto léxico. La sintaxis se evalúa únicamente cuando no existen errores léxicos; los diagnósticos y el informe distinguen las etapas.

Las copias bajo Documentacion/Entregable4 conservan evidencia histórica. Esta memoria y la matriz del entregable 5 describen el estado final verificado. Las diferencias en resultados históricos deben interpretarse según la entrada y la versión utilizada.

Guía de lectura

Las secciones siguientes presentan la especificación final, los reconocedores y la arquitectura. Después se explican la ampliación sintáctica y la validación. El manual de usuario se integra al final como anexo. La matriz detallada se entrega en un PDF independiente, acompañada de archivos de entrada e informes TXT.

Antecedentes y trazabilidad documental

Esta memoria integra la especificacion del entregable 1, el diseno del entregable 2, el motor documentado en el entregable 3 y la integracion del entregable 4. Se actualizaron las reglas y clases segun el codigo final, en lugar de presentar los disenos tempranos como si describieran sin cambios la aplicacion actual.

Fuente original	Contenido recuperado y actualizacion
E1 Especificacion lexica y diseno conceptual	Eleccion de un subconjunto inspirado en C#, 13 reservadas originales, matriz de tokens, cinco reglas de error y atributos de Simbolo. La version final incorpora 30 reservadas predeterminadas y las categorias ampliadas.
E2 Segundo entregable Analizador Lexico	Tabla de transiciones S0 a S10, arquitectura del nucleo, mockup y plan de trabajo. La memoria utiliza estados explicativos consistentes y los nombres de clases actuales.
E3 Entregable3_final.pdf	Motor manual, posiciones base 1, reinicio de resultados, maximo avance, recuperacion y 28 verificaciones originales.
E4 Documentacion conservada	Integracion de editor, carga, tablas, estadisticas, exportacion, manual y 76 verificaciones de robustez.

Trabajo documentado del equipo

La bitacora del entregable 1 atribuye a Jose Roberto la matriz de tokens, a Enrique la especificacion y reservadas, y a Cristian las reglas de error, simbolos y TXT. El entregable 2 asigna para la siguiente etapa lectura y posiciones a Jose Roberto, reconocedores a Enrique y simbolos y pruebas a Cristian. Estas asignaciones historicas no sustituyen la explicacion individual del codigo en la sustentacion.

Version actual y acceso

La version final se encuentra en github.com/CGRM2/C-EXTENDED. El repositorio Zenite50/C-Extended aparece en documentos antiguos y corresponde a etapas previas. Los textos del programa y las pruebas manuales actuales usan ASCII; las reglas de reconocimiento siguen admitiendo letras Unicode y su prueba interna utiliza escapes C# en el archivo fuente.

Resultados historicos y evolucion

El entregable 3 registro cinco archivos y 28 verificaciones aprobadas. La tabla conserva los valores documentados entonces. Se distingue esta evidencia historica de las baterias actuales para no atribuir a una etapa resultados obtenidos despues.

Caso original	Tokens	Simbolos	Errores
prueba1_sencillo	25	3	0
prueba2_medio	62	9	0
prueba3_errores	20	4	4
prueba4_recuperacion	21	4	2
prueba5_categorias	59	7	0

En prueba3 el simbolo @ aparecia despues de una apertura de comentario sin cierre. Por eso formaba parte del comentario y no correspondia informar un quinto error independiente. El cuarto archivo comprobo @ fuera de comentarios y la continuidad de lectura posterior.

Actualizaciones incorporadas

La especificacion original tenia 13 reservadas. El motor amplio ese conjunto y formalizo caracteres literales, operadores logicos y escapes. Se precisaron decimales incompletos, cierres de comentario sin apertura y recuperacion. La arquitectura final conserva Core y UI e incorpora ResultadoAnalisis y AnalizadorSintactico.

La verificacion final ejecutada sobre la version ASCII aprobo 22 casos documentados, 76 comprobaciones lexicas y 125 de sintaxis e integracion. Los 22 casos se ejecutan de manera independiente en el programa de pruebas, aunque sus entradas manuales se entreguen reunidas en un solo TXT.

Organizacion de la evidencia actual

Prueba_Unica_C_Extended.txt contiene bloques P01 a P22. Evidencias/PruebasFinales/Resultados_Consolidados.md conserva cada informe. matriz_resultados.json conserva entradas y diagnosticos completos. El bloque P04 manual utiliza anio para permanecer en ASCII; el caso P04 automatico verifica el identificador Unicode original mediante escapes C#.

Especificacion lexica final

Notación: L representa una letra admitida por char.IsLetter; D representa un dígito ASCII de 0 a 9. El guion bajo se admite al inicio y en la continuación de un identificador. El signo + en las fórmulas significa una o más repeticiones y * significa cero o más. La comparación de nombres es ordinal y sensible a mayúsculas.

Categoría	Regla o conjunto	Ejemplos
TK_RESERVADA	Pertenencia al diccionario como reservada	var, if, return
TK_IDENTIFICADOR	(L|_)(L|D|_)* y clasificación de identificador	total, _dato, año
TK_NUM_ENTERO	D+	0, 10, 2026
TK_NUM_DECIMAL	D+ . D+	0.5, 3.14, 12.0001
TK_CADENA	Comillas dobles, contenido permitido y escapes	"Hola", "", "a\nb"
TK_CARACTER	Comillas simples y exactamente una unidad o escape	'a', 'ñ', '\n'
TK_OP_ARITMETICO	+ - * / % ++ --	+, %, ++
TK_OP_RELACIONAL	== != < > <= >=	==, <=, >
TK_OP_ASIGNACION	= += -= *= /= %=	=, +=, %=
TK_OP_LOGICO	! && ||	!, &&, ||
TK_DELIMITADOR	( ) { } [ ] ; , . :	(, ;, :
TK_COMENTARIO_LINEA	// hasta antes del salto de línea	// nota, // 1, // fin
TK_COMENTARIO_BLOQUE	/* hasta el primer */	/**/, /* nota */, /* a b */

Los números se reconocen como texto; no se convierten a int o double. Un literal largo puede reconocerse sin demostrar que su valor cabe en un tipo numérico de ejecución.

Alfabeto y reglas de error

Reservadas predeterminadas

```
namespace class enum func var nil if else for while
return break continue int float double char bool string void
static true false switch case default public private using new
```

El diccionario puede modificar la clasificación léxica. Los resultados de la matriz se obtuvieron con las clasificaciones disponibles durante la ejecución. Cambiar una palabra no crea producciones de la gramática sintáctica.

Literales y convenciones

Los escapes admitidos son \n, \t, \r, \0, \\, \" y \'. Una cadena no puede atravesar un salto de línea. Un carácter literal contiene exactamente una unidad de texto UTF-16 o un escape reconocido. El programa no normaliza Unicode ni garantiza que un símbolo representado por un par sustituto sea un carácter literal válido.

Las posiciones comienzan en línea 1, columna 1. CRLF cuenta como un salto; CR y LF también separan líneas. La tabulación consume una unidad de columna, por lo que la columna lógica puede diferir de su ancho visual. Una entrada vacía corresponde a la línea 1.

Situación	Diagnóstico y recuperación
12abc	Identificador con inicio numérico; consume toda la secuencia.
1., .5, 1..2	Número decimal mal formado; conserva el separador posterior.
Cadena o carácter sin cierre	Error hasta fin de línea; puede continuar en la siguiente.
'' o 'ab'	El carácter literal no contiene exactamente una unidad.
Escape no admitido	Error de escape del literal completo.
/* sin cierre	Consume hasta EOF y produce un error.
*/ sin apertura	Error de cierre de comentario sin apertura.
@ u otro símbolo fuera del lenguaje	Error localizado y avance para continuar.

Los signos negativo y positivo son operadores separados del número. No se admiten exponentes, sufijos numéricos, hexadecimales ni decimales sin dígitos a ambos lados del punto.

Reconocedores de identificadores y números

El analizador selecciona una rama según el carácter actual. Los estados de las tablas siguientes son etiquetas explicativas de la implementación final: no dependen de la numeración de los diagramas tempranos. La aceptación se realiza cuando el siguiente carácter ya no pertenece al lexema; ese carácter queda disponible para el siguiente ciclo.

Identificadores

Estado	Entrada	Transición o acción
I0	L o _	Consume y pasa a I1.
I1	L, D o _	Consume y permanece en I1.
I1	Otro carácter o EOF	Acepta lexema. Consulta diccionario y registra símbolo si es identificador.

Números

Estado	Entrada	Transición o acción
N0	D	Consume y pasa a N1.
N1	D	Permanece en N1.
N1	Punto	Consume y pasa a N2.
N2	D	Consume y pasa a N3.
N3	D	Permanece en N3.
N1	Separador o EOF	Acepta entero.
N3	Separador o EOF	Acepta decimal.
N2	Ausencia de D	Error decimal y consumo de continuación inválida.
N1 o N3	Letra, _ o punto adicional	Consume secuencia inválida y emite error.

La entrada .5 se identifica antes de la rama general de delimitadores y se rechaza como decimal sin parte entera. La entrada 12abc no produce un entero 12 seguido de abc: se informa una única secuencia inválida.

Prioridades de reconocimiento

Las aperturas // y /* tienen prioridad sobre /. Los operadores de dos caracteres tienen prioridad sobre los de uno, por ejemplo >= sobre >. Este máximo avance evita partir un operador compuesto en símbolos independientes.

Reconocedores de literales y comentarios

Reconocedor	Estados y cierre	Error
Cadena	C0 abre con comilla doble; C1 consume contenido; CE consume un escape; CF acepta al cerrar.	Salto de línea o EOF sin cierre. Escape no admitido o control inválido.
Carácter	K0 abre con comilla simple; K1 cuenta unidades; KE procesa escape; KF acepta al cerrar con cuenta igual a 1.	Cuenta distinta de 1, escape inválido, control inválido o falta de cierre.
Comentario de línea	L0 reconoce //; L1 consume hasta CR, LF o EOF y acepta.	La llegada a EOF es un cierre válido.
Comentario de bloque	B0 reconoce /*; B1 consume; B2 observa *; con / posterior acepta.	EOF sin */ genera un error desde la apertura.

Decisión sobre anidamiento

Los comentarios de bloque no se anidan. Una segunda apertura /* es contenido ordinario y el primer */ cierra el comentario. En P14, el texto posterior se procesa como código y el */ restante genera un error. Esta política es parte del lenguaje definido, no un fallo de la prueba.

```
/* exterior /* interior */ resto */
                              // cierre sobrante
```

Relación entre teoría y código

Cada reconocedor representa un lenguaje regular mediante un número finito de estados y clases de entrada. Su implementación usa condiciones y ciclos en lugar de una tabla universal de transición. AutomataAFD centraliza predicados del alfabeto, reservadas y diccionario; AnalizadorLexico materializa el recorrido y las transiciones.

Las expresiones regulares de especificación describen qué se acepta. El programa no ejecuta esas expresiones para reconocer tokens. Esta distinción permite explicar la equivalencia conceptual entre expresión regular, autómata y reconocedor manual.

Manual técnico y arquitectura final

El proyecto utiliza C#, Windows Forms y .NET Framework 4.8. Core contiene los reconocedores y modelos. UI controla presentación, archivos e interacción. ResultadoAnalisis reúne una instantánea de ambas etapas para evitar que tablas y estadísticas mezclen resultados de textos distintos.

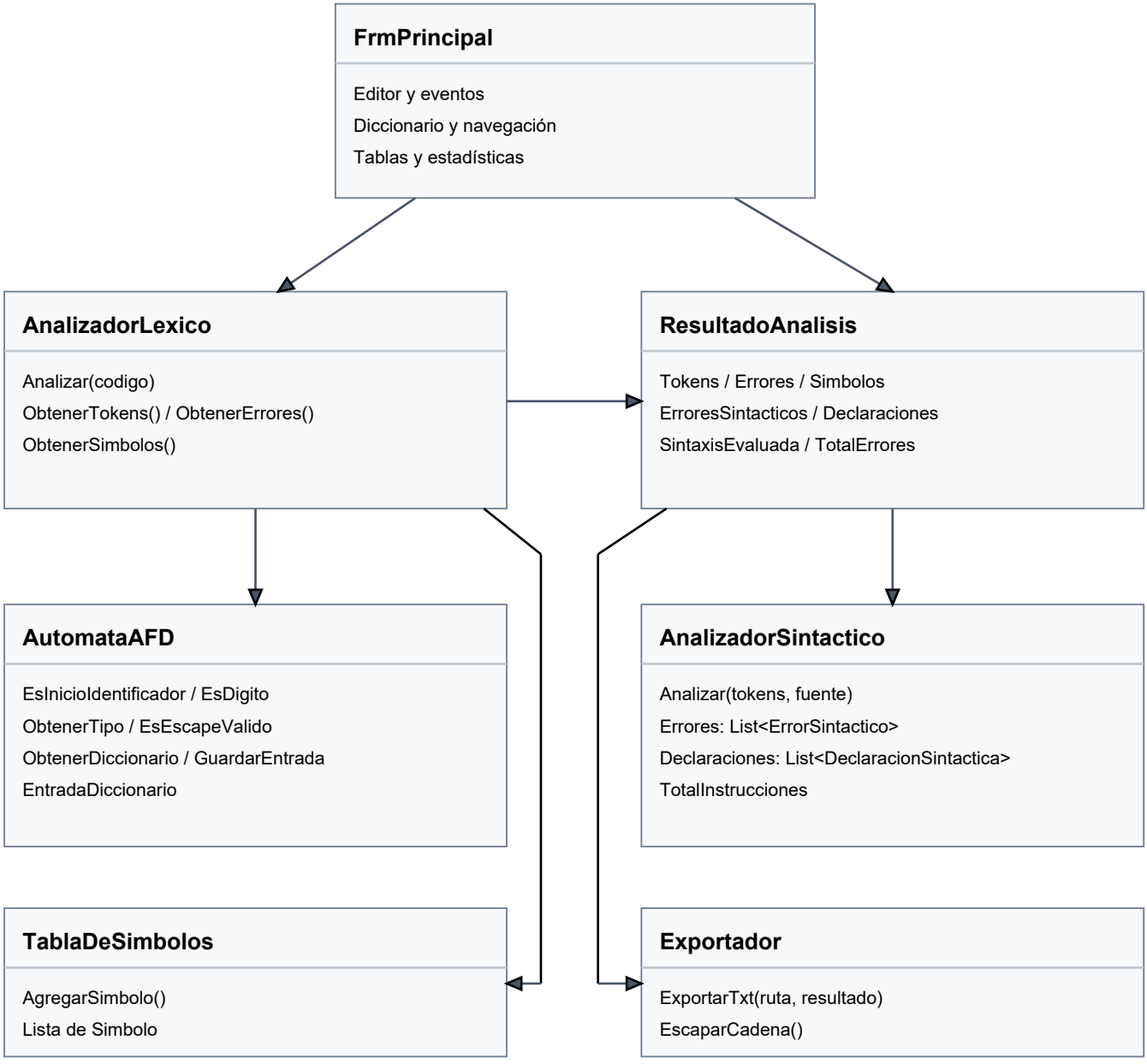
Componente	Responsabilidad
AnalizadorLexico	Analizar, AnalizarArchivo, ObtenerTokens, ObtenerErrores y ObtenerSimbolos. Administra cursor, líneas y recuperación.
AutomataAFD y EntradaDiccionario	Predicados, clasificación ordinal, carga y persistencia del diccionario XML.
Token y ErrorLexico	Lexema/categoría o secuencia inválida/descripción, con línea y columna.
TablaDeSimbolos y Simbolo	Identificadores únicos, orden de aparición y primera ubicación.
AnalizadorSintactico	Consume tokens, reconoce instrucciones y expresiones, informa errores y declaraciones.
ErrorSintactico y DeclaracionSintactica	Ubicación del token encontrado; nombre, tipo y rol de declaraciones reconocidas.
ResultadoAnalisis	Tokens, errores, símbolos, fuente, tiempos, sintaxis evaluada y total de errores.
FrmPrincipal y archivos parciales	Editor, tablas, análisis, diccionario, navegación, atajos y estados visibles.
Exportador	Serialización de la instantánea a un informe TXT con las dos etapas.

Flujo de ejecución

El formulario obtiene el texto, ejecuta el motor léxico y construye ResultadoAnalisis. Si no hay errores léxicos, el resultado ejecuta el analizador sintáctico. El formulario presenta la instantánea y Exportador la escribe cuando el usuario lo solicita. Los comentarios permanecen en la tabla de tokens, pero se filtran antes del análisis sintáctico.

Diagrama de clases actualizado

Diagrama de dependencias de las clases principales. Las flechas indican uso o transferencia de resultados entre los componentes. Los modelos auxiliares se agrupan por su responsabilidad para mantener legibilidad.



ResultadoAnalisis contiene listas de Token, ErrorLexico, Simbolo, ErrorSintactico y DeclaracionSintactica. AutomataAFD administra EntradaDiccionario. El formulario consume estas listas y el exportador consulta la misma instantánea.

Ampliación sintáctica y decisiones de diseño

El analizador sintáctico utiliza descenso recursivo y análisis de expresiones por precedencia. Reconoce declaraciones, funciones, bloques, namespace, class, enum, using, if/else, while, for y switch. Permite sentencias dentro de clases para conservar los ejemplos académicos del proyecto.

Prioridad menor a mayor	Operadores
1 Asignación	= += -= *= /= %=
2 Disyunción	||
3 Conjunción	&&
4 Igualdad	== !=
5 Comparación	< > <= >=
6 Suma y resta	+ -
7 Producto	* / %
Prefijos y accesos	! + - ++ --; llamadas, miembros e índices

Las asignaciones se asocian a la derecha y los operadores binarios restantes se resuelven por prioridad y avance. El parser comprueba destinos asignables en asignaciones e incrementos. Cada función reinicia su contexto de ciclos y switch; return requiere función, break requiere ciclo o switch y continue requiere ciclo.

La recuperación sintáctica busca punto y coma, llave de cierre o cambio de línea. Existen controles de profundidad de 128 niveles y un límite de diagnósticos en los recorridos para evitar procesamientos descontrolados. Las pruebas de regresión verifican estos escenarios.

Límites explícitos

La aplicación no ejecuta código ni es un compilador completo de C#. No verifica compatibilidad de tipos, resolución de ámbitos, variables no declaradas ni nombres duplicados. Las declaraciones presentadas por nombre son información sintáctica y pueden ser parciales si existen errores. La ausencia de diagnósticos acredita únicamente las reglas implementadas.

Pruebas finales y evidencia

Batería	Aprobadas	Fallidas	Evidencia
Casos finales P01 a P22	22	0	matriz_resultados.json e informes consolidados
Regresión léxica	76	0	regresion_lexica_76.txt
Sintaxis e integración	125	0	RegresionSintaxis/ejecucion_125.txt

Los 22 casos verifican lexemas y categorías seleccionadas, cantidad de errores léxicos y estado sintáctico. Para entradas sintácticamente inválidas se exige al menos un diagnóstico; el número obtenido se conserva en la matriz. P08 verifica adicionalmente la ubicación 2:14 del token int después de CRLF y un comentario.

La batería cubre todas las categorías mediante entradas válidas y combina errores en P11. P03 usa un identificador de 1038 caracteres. P02 utiliza muchos decimales sin convertir el literal a un valor de máquina. P14 comprueba la política de comentarios no anidados. Los 22 informes se generaron con el Exportador de la aplicacion y se reunieron en Resultados_Consolidados.md.

Cómo reproducir

```
msbuild C-Extended.csproj /p:Configuration=Release
msbuild PruebasFinales/PruebasFinales.csproj /p:Configuration=Release
PruebasFinales/bin/Release/PruebasFinales.exe evidencia_final

msbuild PruebasConsola/PruebasConsola.csproj /p:Configuration=Release
PruebasConsola/bin/Release/PruebasConsola.exe Pruebas

msbuild PruebasSintaxis/PruebasSintaxis.csproj /p:Configuration=Release
PruebasSintaxis/bin/Debug/PruebasSintaxis.exe evidencia_sintaxis
```

Ejecutar desde una consola de desarrollador de Visual Studio, en la raíz del código. PruebasSintaxis conserva la carpeta bin/Debug incluso con Configuration=Release. Las pruebas de integración abren un formulario brevemente y generan capturas. No se sustituyen resultados fallidos por valores esperados.

Correcciones y pendientes de validación

La base verificada ya incorpora correcciones de arreglos propios, contexto de funciones, modificadores y protección de la fuente al exportar. No fue necesario cambiar el motor para aprobar la matriz final. La demostración ante el docente y la prueba en otra computadora son verificaciones presenciales pendientes; las pruebas locales no certifican su ejecución.

Manual de usuario Instalación y ejecución

Requisitos

Computadora con Windows y .NET Framework 4.8. Para compilar se necesita Visual Studio con Desarrollo de escritorio de .NET y el paquete de destino de .NET Framework 4.8. La aplicación final no utiliza base de datos ni requiere .NET 10 para ejecutarse.

Ejecutar la versión compilada

Extraer completamente el paquete de entrega en una carpeta con permiso de lectura.

Abrir la carpeta Aplicacion y ejecutar C-Extended.exe. Conservar C-Extended.exe.config y Manual_de_Uso.txt junto al ejecutable.

Comprobar que aparecen el editor, los botones superiores y las pestañas de resultados.

Cargar un archivo TXT de la carpeta de pruebas o escribir código en el editor.

Compilar desde el código

Abrir C-Extended.csproj en Visual Studio, seleccionar Release y compilar. Ejecutar bin/Release/C-Extended.exe. Si falta el paquete de destino, instalar el componente correspondiente desde Visual Studio Installer antes de repetir la compilación.

Archivos y codificación

La interfaz admite UTF-8 con o sin BOM y UTF-16 con BOM. Rechaza UTF-32 y secuencias inválidas para evitar cambiar silenciosamente el código. Si un archivo no carga, guardarlo nuevamente como texto UTF-8 y volver a abrirlo.

Antes de presentar

Probar el ejecutable extraído en el equipo de la exposición, copiar el bloque P01 de Prueba_Unica_C_Extended.txt, revisar que no aparezcan errores y copiar el bloque P19 para observar un diagnóstico sintáctico. Si se ha personalizado el diccionario, comprobar que conserva las clasificaciones que utiliza la demostración. No modificar las reservadas justo antes de presentar.

Manual de usuario Analizar y leer tokens

1. Pulsar Cargar archivo o usar Ctrl+O para abrir un TXT. También puede escribirse código en el editor.
2. Pulsar Analizar o F5. El análisis automático se ejecuta tras una pausa aproximada de 400 ms al editar.
3. Abrir Tokens y comparar lexema, categoría, línea y columna.

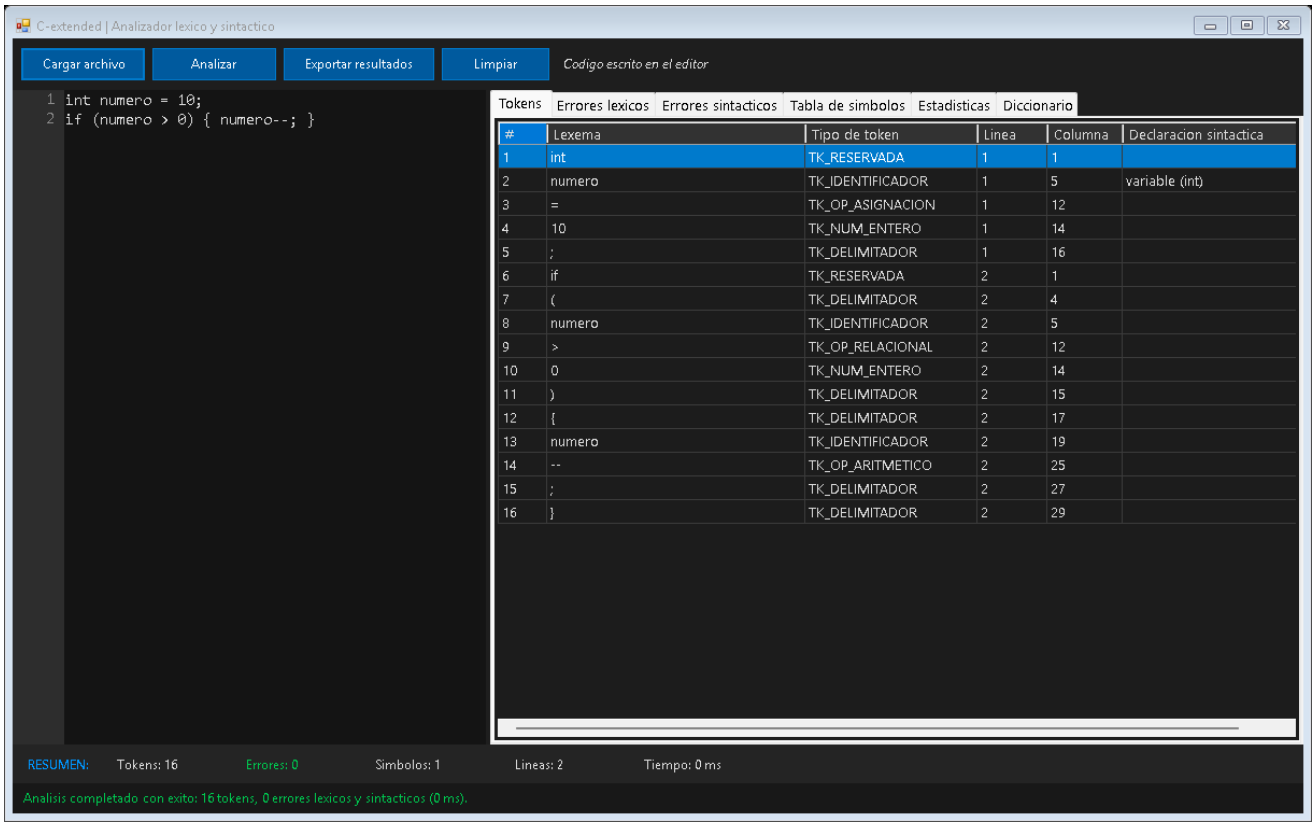


Figura 1. Captura real de la versión verificada: declaración y condición sin errores.

Cada fila representa un token. La columna Declaración sintáctica muestra el rol reconocido de los identificadores declarados. Un doble clic en una fila lleva a su ubicación en el editor. Las posiciones empiezan en 1; el número de fila de la tabla no es la posición del carácter.

```
int numero = 10;
if (numero > 0) { numero--; }
```

Esta entrada produce 16 tokens y un identificador único. numero aparece varias veces en tokens y una sola vez en la tabla de símbolos. El resumen de errores corresponde a ambas etapas.

Manual de usuario Corregir errores

Abrir primero Errores léxicos. Corregir las secuencias indicadas y volver a analizar. Si hay errores léxicos, la sintaxis queda PENDIENTE: una tabla sintáctica vacía en ese estado no implica que el programa sea correcto.

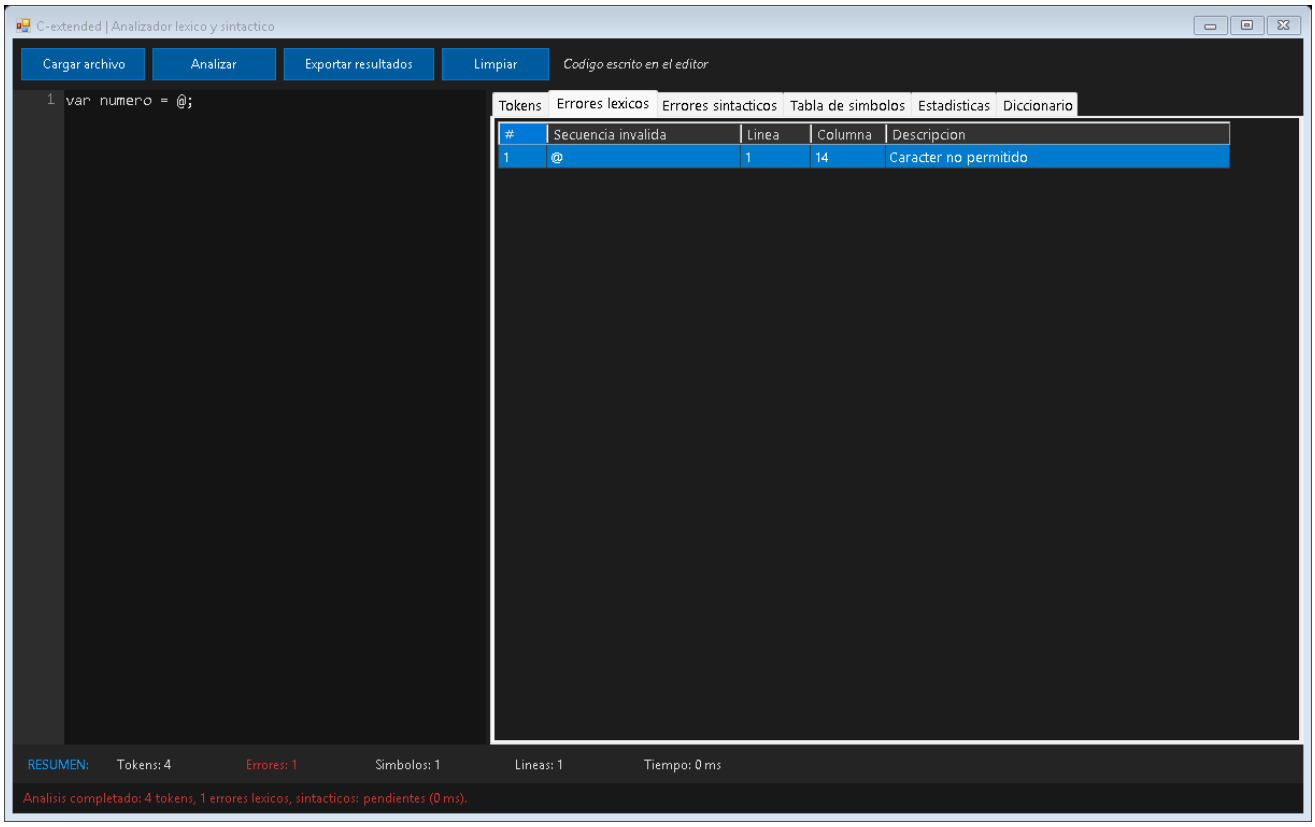


Figura 2. El símbolo @ genera un error léxico y deja pendiente la etapa sintáctica.

Cuando la etapa léxica no encuentra errores, consultar Errores sintácticos. La descripción indica lo esperado y Encontrado muestra el token donde se detectó el problema. Un símbolo faltante puede notificarse en el token siguiente o al final del archivo. Corregir primero el primer diagnóstico para reducir errores derivados.

Ejemplos: var x = @; es un error léxico. var x = ; contiene tokens válidos, pero falta la expresión y produce un error sintáctico. Los límites del lenguaje se explican en el botón Ayuda y en Manual_de_Uso.txt.

Manual de usuario Sintaxis y navegación

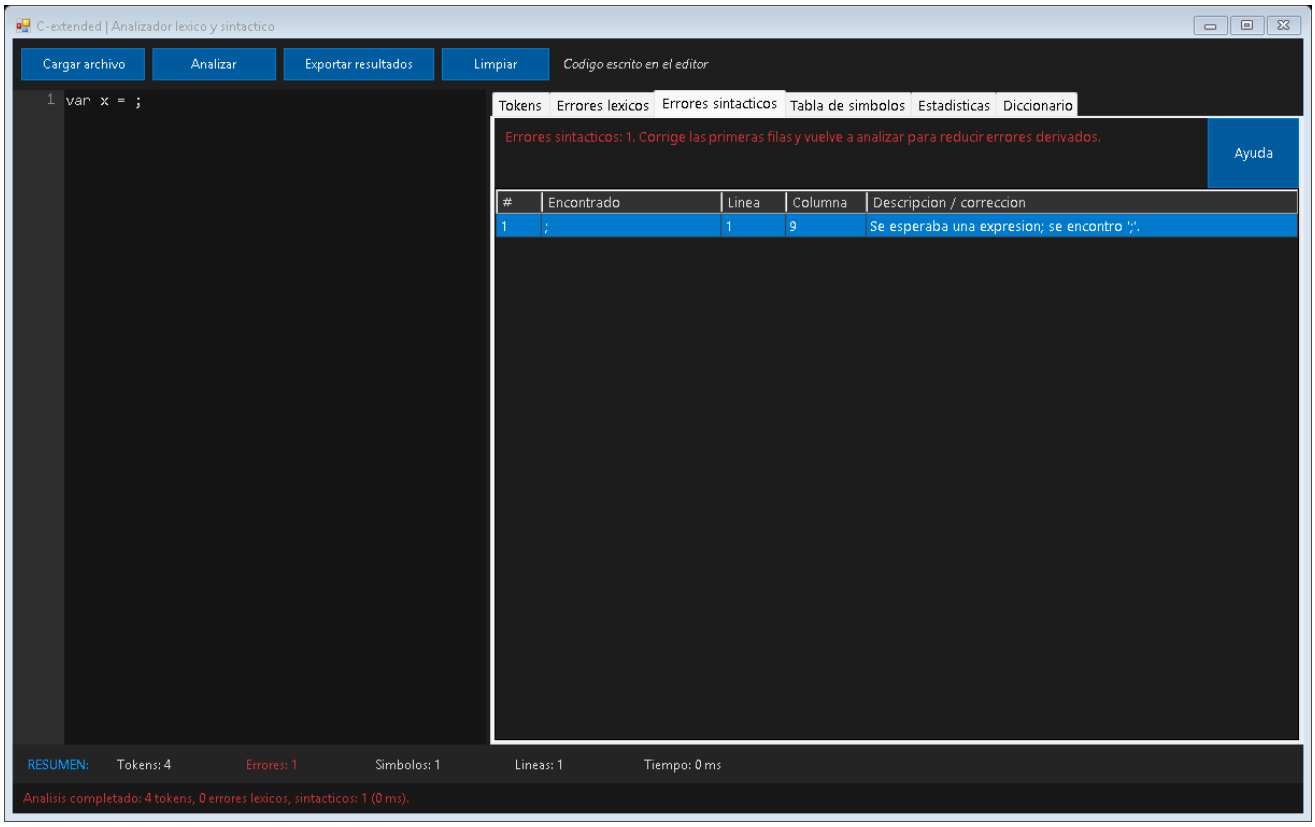


Figura 3. Captura real de var x = ;. Se esperaba una expresión en línea 1, columna 9.

Hacer doble clic en el diagnóstico para seleccionar el token en el editor. Si el error apunta a EOF, la selección se acota al final del texto. Corregir la entrada, por ejemplo var x = 1;, y pulsar F5 para actualizar ambas etapas.

Limpiar y preservar el trabajo

Limpiar vacía el editor y los resultados e impide exportar el análisis anterior. La aplicación no guarda automáticamente el código escrito. Conservar una copia del archivo fuente o copiar el texto antes de limpiar o cerrar. Exportar crea un informe de resultados, no un archivo de código listo para recuperar la sesión.

Atajos

Atajo	Acción
Ctrl+O	Cargar archivo de texto
F5	Analizar el código actual
Ctrl+E	Exportar el informe cuando existe un resultado vigente

Manual de usuario Símbolos y estadísticas

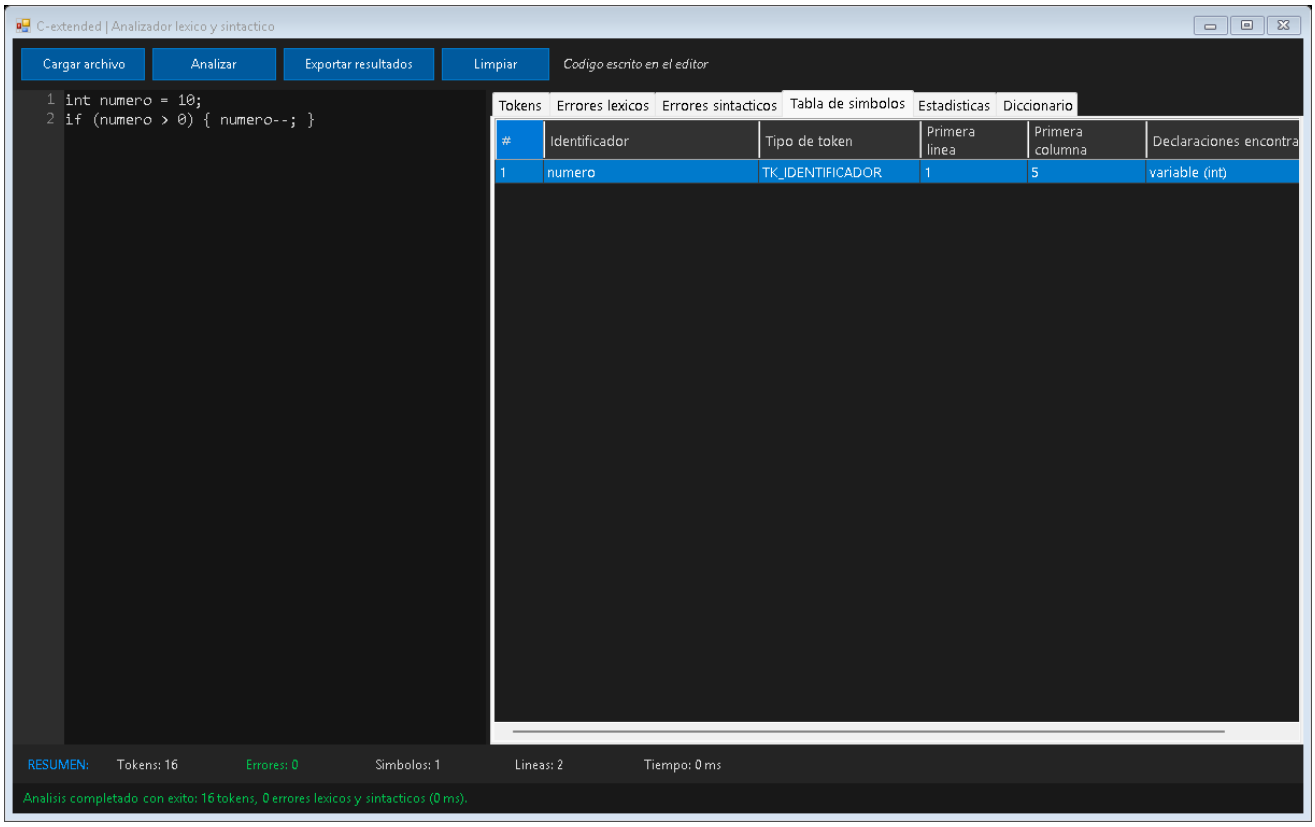


Figura 4. La tabla conserva identificadores únicos y su primera aparición.

La tabla de símbolos agrupa por nombre y conserva la primera línea y columna. Las declaraciones asociadas informan los roles encontrados por la etapa sintáctica. No representan una resolución de ámbitos: dos variables con el mismo nombre en distintos bloques no se administran como símbolos semánticos independientes.

Estadísticas muestra tokens, identificadores, errores por etapa, declaraciones, líneas y tiempos. Cuando la sintaxis no se evaluó, el estado debe leerse como pendiente. Los milisegundos pueden ser cero en entradas pequeñas; el valor depende del reloj y de la computadora, y no equivale a una medición de rendimiento exhaustiva.

Interpretación del resumen

Errores totales suma los diagnósticos de las etapas evaluadas. Si la sintaxis quedó pendiente, ese total no permite afirmar que no existan errores sintácticos. El texto de estado y las pestañas aclaran qué etapas se ejecutaron.

Manual de usuario Diccionario y exportación

Administrar el diccionario

Abrir Diccionario. Para agregar una entrada, pulsar Nuevo, escribir una palabra válida, elegir TK_RESERVADA o TK_IDENTIFICADOR, completar la descripción y pulsar Agregar. Para modificar, seleccionar la fila y pulsar Modificar después de editar. Para eliminar, seleccionar la fila, pulsar Eliminar y confirmar.

No se permiten duplicados, espacios ni nombres con inicio numérico. Los cambios persisten en %LOCALAPPDATA%/C-Extended/diccionario.xml. Las palabras se comparan distinguiendo mayúsculas. Agregar una reservada clasifica el token, pero no añade instrucciones al parser.

Exportar

Analizar la entrada y pulsar Exportar resultados o Ctrl+E. Seleccionar un nombre TXT diferente al archivo fuente. La protección de destino evita sobrescribir el archivo cargado. El informe incluye tokens, errores léxicos y sintácticos, símbolos, declaraciones, estado de las etapas y tiempos.

```
Ejemplo de entrada: var x = ;
Resultado: 0 errores léxicos y 1 error sintáctico
Ubicación: línea 1, columna 9
Descripción: se esperaba una expresión
```

Si se edita el código, esperar al análisis automático o pulsar F5 antes de exportar. Si hay errores léxicos, el informe señala que la sintaxis está pendiente y que las declaraciones no se evaluaron. Los saltos y caracteres especiales se representan de forma escapada para conservar la legibilidad de las filas.

Solución de problemas

Situación	Acción
No inicia	Verificar .NET Framework 4.8 y extraer el paquete completo.
TXT rechazado	Guardar nuevamente en UTF-8; no usar UTF-32.
Sintaxis pendiente	Corregir primero todos los errores léxicos.
Exportar deshabilitado	Analizar el texto actual; no reutilizar resultados de antes de limpiar.
Una reservada nueva falla	Restaurar su clasificación o usar una construcción con regla existente.

Conclusiones y trabajo futuro

El proyecto integra una especificación léxica con reconocedores manuales y una interfaz que muestra resultados localizables. Las categorías, las posiciones y las reglas de recuperación se verificaron sobre entradas representativas y casos límite. La ampliación sintáctica mantiene separados los errores y evita analizar secuencias léxicamente inválidas como si fueran programas completos.

La matriz final documenta 22 casos aprobados y conserva sus entradas e informes. Las baterías de regresión añadieron 201 comprobaciones aprobadas. La documentación actualiza las reglas de números, comentarios, literales y reservadas para que coincidan con el código revisado.

Trabajo futuro

Se propone incorporar resolución de ámbitos, verificación de tipos y referencias a declaraciones como una etapa semántica independiente. También convendría administrar perfiles de diccionario reproducibles y añadir exportación estructurada para herramientas externas. Estas funciones se proponen como ampliaciones y no se presentan como implementadas.

Uso de herramientas de apoyo

Se utilizó Codex como apoyo para revisar código, preparar y ejecutar pruebas y consolidar la documentación. Los resultados publicados corresponden a ejecuciones reales. Todos los integrantes deben revisar la implementación y poder explicar las decisiones presentadas durante la sustentación.

Fuentes y trazabilidad

Guía docente: Proyecto_Analizador_Lexico_CSharp_WinForms.pdf, especialmente páginas 14 y 15. Antecedentes: especificación del entregable 1, diseño del entregable 2, informe del entregable 3 y documentación conservada en Documentacion/Entregable4.

Fuente técnica principal: Core y UI del repositorio CGRM2/C-EXTENDED, commit 88216455dc6a2ab224b3affb575a1258786b3efe. La carpeta PruebasFinales incorpora el ejecutor de los casos P01 a P22. Evidencia detallada: Matriz_Pruebas_Entregable5.pdf, Prueba_Unica_C_Extended.txt e informes consolidados de la carpeta Evidencias.